

[例 5.4] 显式说明参照完整性的违约处理示例。

```
CREATE TABLE SC
(
    Sno CHAR(8),
    Cno CHAR(5),
    Grade SMALLINT,          /* 成绩 */
    Semester CHAR(5),        /* 开课学期 */
    Teachingclass CHAR(8),    /* 学生选修某一门课所在的教学班 */
    PRIMARY KEY(Sno,Cno), /* 在表级定义实体完整性,Sno、Cno 都不能取空值 */
    FOREIGN KEY(Sno) REFERENCES Student(Sno) /* 在表级定义参照完整性 */
    ON DELETE CASCADE /* 当删除 Student 表中的元组时, */
                        /* 级联删除 SC 表中 Sno 与 Student.Sno 相同的 SC 元组 */
    ON UPDATE CASCADE, /* 当更新 Student 表中的 Sno 时, */
                        /* 级联更新 SC 表中相应元组的 Sno */
    FOREIGN KEY(Cno) REFERENCES Course(Cno)
                        /* 在表级定义参照完整性 */
    ON DELETE NO ACTION /* 当删除 Course 表中的元组造成与 SC 表不一致,即 SC 中存在
                        /* 这样的元组,其 Cno 与 Course.Cno 相同,则拒绝删除 Course
                        /* 表的这个元组 */
    ON UPDATE CASCADE /* 当更新 Course 表中的 Cno 时, */
                        /* 级联更新 SC 表中相应元组的 Cno */
);
```

可以对 DELETE 和 UPDATE 采用不同的策略。例如,例 5.4 中当删除被参照表 Course 表中的元组,造成与参照表(SC 表)不一致时,拒绝删除被参照表的元组;对更新操作则采用级联更新的策略。

从上面的讨论可以看到,关系数据库管理系统在实现参照完整性时,除了要提供定义主码、外码的机制外,还需要提供不同的策略供用户选择。具体选择哪种策略,要根据应用环境的要求确定。

5.4 用户定义的完整性

用户定义的完整性就是某一具体应用涉及的数据必须满足的语义要求。目前的关系数据库管理系统都提供了定义和检查这类完整性的机制,使用和实体完整性、参照完整性相同的技术和方法来处理它们,而不必由应用程序实现这一功能。

5.4.1 属性上的约束

1. 属性上约束的定义

在 CREATE TABLE 中定义属性的同时,可以根据应用要求定义属性上的约束,即属性值限制,包括:

- ① 列值非空(NOT NULL)。
- ② 列值唯一(UNIQUE)。
- ③ 检查列值是否满足一个条件表达式(CHECK 短语)。

2. 不允许取空值

[例 5.5]在定义 SC 表时,说明 Sno、Cno、Grade 属性不允许取空值。

```
CREATE TABLE SC
(Sno CHAR(8) NOT NULL,      /* Sno 属性不允许取空值 */
 Cno CHAR(5) NOT NULL,      /* Cno 属性不允许取空值 */
 Grade SMALLINT NOT NULL,   /* Grade 属性不允许取空值 */
 Semester CHAR(5),
 Teachingclass CHAR(8),
 PRIMARY KEY(Sno, Cno),     /* 在表级定义实体完整性,隐含 Sno、Cno 不允许 */
                               /* 取空值,在列级不允许取空值的定义可不写 */
);
```

3. 列值唯一

[例 5.6]建立学院表 School,要求学院名称 SHname 列取值唯一,学院编号 SHno 列为主码。

```
CREATE TABLE School
(SHno CHAR(8) PRIMARY KEY,   /* SHno 列为主码 */
 SHname VARCHAR(40) UNIQUE, /* 要求 SHname 值唯一 */
 SHfounddate Date           /* 学院创建日期 */
);
```

4. 用 CHECK 短语指定列值应该满足的条件

[例 5.7]Student 表的 Ssex 属性只允许取“男”或“女”。

```
CREATE TABLE Student
(Sno CHAR(8) PRIMARY KEY,    /* 在列级定义主码 */
 Sname CHAR(20) NOT NULL,    /* Sname 属性不允许取空值 */
 Ssex CHAR(6) CHECK(Ssex IN('男','女')), /* 性别属性 Ssex 只允许取'男'或'女' */
 Sbirthdate Date,
```

```
Smajor VARCHAR(40)
);
```

[例 5.8] SC 表 Grade 的值应该在 0~100。

```
CREATE TABLE SC
(Sno CHAR(8),
Cno CHAR(5),
Grade SMALLINT CHECK (Grade >= 0 AND Grade <= 100), /* Grade 取值范围 0~100 */
Semester CHAR(5),
Teachingclass CHAR(8),
PRIMARY KEY(Sno, Cno),
FOREIGN KEY(Sno) REFERENCES Student(Sno),
FOREIGN KEY(Cno) REFERENCES Course(Cno)
);
```

5. 属性上约束的检查和违约处理

当往表中插入元组或修改属性的值时，关系数据库管理系统将检查属性上的约束是否被满足，如果不满足则操作被拒绝执行。

5.4.2 元组上的约束

1. 元组上约束的定义

与属性上约束的定义类似，在 CREATE TABLE 语句中可以用 CHECK 短语定义元组上的约束，即元组级的限制。同属性值限制相比，元组级的限制可以设置不同属性之间取值的相互约束。

[例 5.9] 当学生的性别是“男”时，其姓名不能以 Ms. 打头。

```
CREATE TABLE Student
(Sno CHAR(8),
Sname CHAR(20) NOT NULL,
Ssex CHAR(6),
Sbirthdate Date,
Smajor VARCHAR(40),
PRIMARY KEY(Sno),
CHECK(Ssex='女' OR Sname NOT LIKE 'Ms. %')
); /* 定义了元组中 Sname 和 Ssex 两个属性值之间的约束条件 */
```

性别是女性的元组都能通过该项 CHECK 检查，因为 Ssex='女'成立；当性别是男性时，要通过检查则姓名一定不能以'Ms.'打头，因为 Ssex='男'时，条件要想为真值，Sname NOT LIKE 'Ms. %' 必须为真值。

2. 元组上约束的检查和违约处理

当往表中插入元组或修改属性的值时，关系数据库管理系统将检查元组上的约束是否被满足，如果不满足则操作被拒绝执行。

5.5 完整性约束命名子句

以上讲解的完整性约束都是在 CREATE TABLE 语句中定义的。SQL 还在 CREATE TABLE 语句中提供了完整性约束命名子句 CONSTRAINT，用来对完整性约束命名，从而可以灵活地增加、删除一个完整性约束。

1. 完整性约束命名子句格式

CONSTRAINT <完整性约束名> <完整性约束>

<完整性约束>包括 NOT NULL、UNIQUE、PRIMARY KEY、FOREIGN KEY、CHECK 短语等。

[例 5.10]建立“学生”表 Student，要求学号在 10000000 到 29999999 之间，姓名不能取空值，出生日期在 1980 年之后，性别只能是“男”或“女”。

```
CREATE TABLE Student
( Sno CHAR(8)
  CONSTRAINT C1 CHECK(Sno BETWEEN '10000000' AND '29999999'),
  Sname CHAR(20)
  CONSTRAINT C2 NOT NULL,
  Sbirthdate Date
  CONSTRAINT C3 CHECK(Sbirthdate >'1980-1-1'),
  Ssex CHAR(6)
  CONSTRAINT C4 CHECK(Ssex IN('男','女')),
  Smajor VARCHAR(40),
  CONSTRAINT StudentKey PRIMARY KEY(Sno)
);
```

在 Student 表上建立了 5 个约束条件，包括主码约束(命名为“StudentKey”)和 C1、C2、C3、C4 这 4 个列级约束。

[例 5.11]建立“教师”表 Teacher，要求每个教师的应发工资(每月)不低于 3 000 元。应发工资是工资列 Sal 与扣除项 Deduct 之和。

```
CREATE TABLE Teacher
( Eno CHAR(8) PRIMARY KEY,    /* 在列级定义主码 */
  Ename VARCHAR(20),
```